Question 1: What is the role of filters and feature maps in Convolutional Neural Network (CNN)?

In a **Convolutional Neural Network (CNN)**, **filters (kernels)** and **feature maps** are core components that enable the model to automatically extract meaningful patterns from input data.

- **Filters (Kernels):** Small learnable matrices that slide over the input image to detect local patterns such as edges, corners, and textures. Each filter focuses on a specific type of feature and its values are learned during training.
- **Feature Maps:** The outputs produced after applying filters across the input. Each feature map highlights where a particular feature appears in the image, preserving spatial relationships.

In essence, filters act as **pattern detectors**, and feature maps represent the **presence and location of those patterns**, enabling CNNs to build hierarchical visual understanding.

Question 2: Explain the concepts of padding and stride in CNNs(Convolutional Neural Network). How do they affect the output dimensions of feature maps?

In **Convolutional Neural Networks (CNNs)**, **padding** and **stride** control how the convolution operation moves over the input and directly influence the **size of the output feature maps**.

Padding: Padding adds extra pixels (usually zeros) around the input image.

- **Purpose:** Prevents loss of information at image borders and helps control output size.
- **Effect:**
  - *Valid padding* (no padding) → output size decreases.
  - *Same padding* → output size remains the same as input.

Stride: Stride defines the step size with which the filter moves across the input.

- **Purpose:** Controls spatial downsampling.
- **Effect:**
  - Larger stride → smaller output feature map.

  - Smaller stride → larger, more detailed feature map.

Output size formula:

$$\text{Output} = \frac{(N - F + 2P)}{S} + 1$$

where (N) = input size, (F) = filter size, (P) = padding, and (S) = stride.

In summary, padding preserves spatial information, while stride controls resolution and computational efficiency.

Question 3: Define receptive field in the context of CNNs. Why is it important for deep architectures?

In **Convolutional Neural Networks (CNNs)**, the **receptive field** refers to the **region of the input image that influences a particular neuron or feature map activation**.

Importance in deep architectures:

- As network depth increases, the **effective receptive field grows**, allowing neurons to capture **larger and more complex patterns**.
- Early layers focus on **local features** (edges, textures), while deeper layers learn **global structures** (objects, shapes).
- A larger receptive field enables better **context understanding**, which is crucial for tasks like image classification and object detection.

In essence, receptive fields allow CNNs to build hierarchical feature representations from local to global information.

Question 4: Discuss how filter size and stride influence the number of parameters in a CNN.

In a **Convolutional Neural Network (CNN)**, **filter size** and **stride** affect the model's **parameter count and computational cost**, though in different ways.

Filter size:

- Larger filters (e.g., 5×5 vs 3×3) have **more trainable parameters** per filter.
- Number of parameters per filter = (F \times F \times C) (where (C) is input channels).
- Larger filters capture wider context but increase **model complexity and risk of overfitting**.

Stride:

- Stride does **not change the number of parameters**, since weights remain the same.

- Larger stride reduces the **number of feature map activations**, lowering computation and memory usage.

Summary: Filter size directly increases parameters and model capacity, while stride controls spatial resolution and efficiency without affecting parameter count.

Question 5: Compare and contrast different CNN-based architectures like LeNet, AlexNet, and VGG in terms of depth, filter sizes, and performance.

Below is a **clear comparison** of **LeNet, AlexNet, and VGG** based on **architecture depth, filter design, and performance**

Comparison of CNN Architectures

Aspect	LeNet	AlexNet	VGG
Introduced	1998	2012	2014
Depth	Shallow (≈5 layers)	Medium (8 layers)	Deep (16–19 layers)
Filter Sizes	Large (5×5)	Mixed (11×11, 5×5, 3×3)	Uniform small (3×3)
Activation Function	Tanh / Sigmoid	ReLU	ReLU
Pooling	Average Pooling	Max Pooling	Max Pooling
Parameters	Very few	~60 million	Very high (~138 million)
Performance	Simple digit recognition	ImageNet breakthrough	High accuracy, heavy computation
Use Case	MNIST-like tasks	Large-scale image classification	Feature extraction, transfer learning

Key Takeaways

- **LeNet** is lightweight and suited for simple tasks.
- **AlexNet** introduced deep learning success with ReLU and GPUs.
- **VGG** emphasized depth using small filters, improving accuracy at the cost of computation.

Question 6: Using keras, build and train a simple CNN model on the MNIST dataset from scratch. Include code for module creation, compilation, training, and evaluation.

```
# --- Suppress TensorFlow Warnings ---
import os, warnings
```

```python
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
warnings.filterwarnings("ignore")

# --- Import Libraries ---
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
from tensorflow.keras.utils import to_categorical

# --- Load MNIST Dataset ---
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# --- Preprocess Data ---
X_train = X_train.reshape(-1, 28, 28, 1) / 255.0
X_test = X_test.reshape(-1, 28, 28, 1) / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# --- Build CNN Model ---
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# --- Compile Model ---
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# --- Train Model ---
model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
```

```
# --- Evaluate Model ---
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 112ms/step - accuracy: 0.8566 - loss:
0.5072 - val_accuracy: 0.9813 - val_loss: 0.0673
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 47s 112ms/step - accuracy: 0.9810 - loss:
0.0614 - val_accuracy: 0.9872 - val_loss: 0.0461
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 42s 99ms/step - accuracy: 0.9867 - loss:
0.0413 - val_accuracy: 0.9847 - val_loss: 0.0555
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 42s 99ms/step - accuracy: 0.9899 - loss:
0.0314 - val_accuracy: 0.9888 - val_loss: 0.0418
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 43s 102ms/step - accuracy: 0.9922 - loss:
0.0239 - val_accuracy: 0.9895 - val_loss: 0.0385
Test Accuracy: 0.9909
```

Question 7: Load and preprocess the CIFAR-10 dataset using Keras, and create a CNN model to classify RGB images. Show your preprocessing and architecture.

```
# --- Suppress TensorFlow Warnings ---
import os, warnings
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
warnings.filterwarnings("ignore")

# --- Import Libraries ---
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras.utils import to_categorical

# --- Load CIFAR-10 Dataset ---
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# --- Preprocessing ---
```

```python
# Normalize pixel values (0–255 → 0–1)
X_train = X_train / 255.0
X_test = X_test / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

print("Training data shape:", X_train.shape)
print("Test data shape:", X_test.shape)

# --- Build CNN Model ---
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(32, 32, 3)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

# --- Compile Model ---
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# --- Model Summary ---
model.summary()

# --- Train Model ---
model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

# --- Evaluate Model ---
```

```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Training data shape: (50000, 32, 32, 3)
Test data shape: (10000, 32, 32, 3)

Model: "sequential_1"

┏━━━━━━━━━━━━━━━━━━━
```

```
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                      │ (None, 10)                   │
2,570 │
└──────────────────────────────────────┴──────────────────────────────┴──────────
──────┘

 Total params: 620,362 (2.37 MB)

 Trainable params: 620,362 (2.37 MB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 101s 283ms/step - accuracy: 0.2985 -
loss: 1.8807 - val_accuracy: 0.5108 - val_loss: 1.3308
Epoch 2/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 99s 282ms/step - accuracy: 0.5259 - loss:
1.3200 - val_accuracy: 0.6266 - val_loss: 1.0567
Epoch 3/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 100s 284ms/step - accuracy: 0.6076 -
loss: 1.1127 - val_accuracy: 0.6780 - val_loss: 0.9140
Epoch 4/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 101s 287ms/step - accuracy: 0.6642 -
loss: 0.9609 - val_accuracy: 0.6892 - val_loss: 0.9055
Epoch 5/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 141s 285ms/step - accuracy: 0.6940 -
loss: 0.8764 - val_accuracy: 0.7254 - val_loss: 0.8018
Epoch 6/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 144s 290ms/step - accuracy: 0.7223 -
loss: 0.7904 - val_accuracy: 0.7332 - val_loss: 0.7844
Epoch 7/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 140s 284ms/step - accuracy: 0.7506 -
loss: 0.7171 - val_accuracy: 0.7530 - val_loss: 0.7267
Epoch 8/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 143s 288ms/step - accuracy: 0.7658 -
loss: 0.6688 - val_accuracy: 0.7548 - val_loss: 0.7178
Epoch 9/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 140s 282ms/step - accuracy: 0.7859 -
loss: 0.6137 - val_accuracy: 0.7650 - val_loss: 0.7045
Epoch 10/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 144s 289ms/step - accuracy: 0.7988 -
loss: 0.5701 - val_accuracy: 0.7670 - val_loss: 0.6816
Test Accuracy: 0.7508
```

Question 8: Using PyTorch, write a script to define and train a CNN on the MNIST dataset. Include model definition, data loaders, training loop, and accuracy evaluation.

```
# --- Import Libraries ---
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# --- Device Configuration ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# --- Data Preprocessing ---
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# --- Load MNIST Dataset ---
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    transform=transform,
    download=True
)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# --- Define CNN Model ---
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.fc1 = nn.Linear(64 * 5 * 5, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = CNN().to(device)

# --- Loss Function and Optimizer ---
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# --- Training Loop ---
epochs = 5

for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}], Loss:
{running_loss/len(train_loader):.4f}")

# --- Evaluation ---
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
accuracy = 100 * correct / total
print(f"Test Accuracy: {accuracy:.2f}%")
```

```
Using device: cpu

100%|██████████| 9.91M/9.91M [00:00<00:00, 135MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 33.4MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 63.9MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.66MB/s]

Epoch [1/5], Loss: 0.1528
Epoch [2/5], Loss: 0.0460
Epoch [3/5], Loss: 0.0319
Epoch [4/5], Loss: 0.0243
Epoch [5/5], Loss: 0.0169
Test Accuracy: 99.01%
```

Question 9: Given a custom image dataset stored in a local directory, write code using Keras ImageDataGenerator to preprocess and train a CNN model.

```
import tensorflow as tf
import tensorflow_datasets as tfds

# Load dataset directly with TensorFlow Datasets
(ds_train, ds_val), ds_info = tfds.load(
    'cats_vs_dogs',
    split=['train[:80%]', 'train[80%:90%]'],
    as_supervised=True,
    with_info=True
)

def preprocess(image, label):
    image = tf.image.resize(image, (150,150)) / 255.0
    return image, label

train_dataset = ds_train.map(preprocess).batch(32)
val_dataset = ds_val.map(preprocess).batch(32)

# Build model (same as before)
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation="relu",
input_shape=(150,150,3)),
    tf.keras.layers.MaxPool2D(2,2),
    tf.keras.layers.Conv2D(64, (3,3), activation="relu"),
    tf.keras.layers.MaxPool2D(2,2),
```

```
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_dataset, epochs=5,
validation_data=val_dataset)

val_loss, val_acc = model.evaluate(val_dataset)
print(f"Validation Accuracy: {val_acc:.4f}")
```

```
WARNING:absl:Variant folder
/root/tensorflow_datasets/cats_vs_dogs/4.0.1 has no dataset_info.json

Downloading and preparing dataset Unknown size (download: Unknown
size, generated: Unknown size, total: Unknown size) to
/root/tensorflow_datasets/cats_vs_dogs/4.0.1...

{"model_id":"8c88130ecb61453a99e3d2e131c63000","version_major":2,"vers
ion_minor":0}

{"model_id":"8c0f7ac8fee245b690fa453127cf995e","version_major":2,"vers
ion_minor":0}

{"model_id":"b5f5ea7ce25843bda3e9956b273d970a","version_major":2,"vers
ion_minor":0}

{"model_id":"bd72c80c357a4c85966246e42c826895","version_major":2,"vers
ion_minor":0}

WARNING:absl:1738 images were corrupted and were skipped

{"model_id":"b99f683995df4de5bea158c09ebcb878","version_major":2,"vers
ion_minor":0}

Dataset cats_vs_dogs downloaded and prepared to
/root/tensorflow_datasets/cats_vs_dogs/4.0.1. Subsequent calls will
reuse this data.
Epoch 1/5
582/582 ━━━━━━━━━━━━━━━━━━━━ 732s 1s/step - accuracy: 0.6051 - loss:
0.7218 - val_accuracy: 0.7068 - val_loss: 0.5837
Epoch 2/5
582/582 ━━━━━━━━━━━━━━━━━━━━ 720s 1s/step - accuracy: 0.7270 - loss:
0.5486 - val_accuracy: 0.7369 - val_loss: 0.5273
Epoch 3/5
582/582 ━━━━━━━━━━━━━━━━━━━━ 719s 1s/step - accuracy: 0.7950 - loss:
0.4433 - val_accuracy: 0.7584 - val_loss: 0.5222
```

```
Epoch 4/5
582/582 ━━━━━━━━━━━━━━━━━━━━ 728s 1s/step - accuracy: 0.8552 - loss:
0.3341 - val_accuracy: 0.7653 - val_loss: 0.5561
Epoch 5/5
582/582 ━━━━━━━━━━━━━━━━━━━━ 741s 1s/step - accuracy: 0.8939 - loss:
0.2462 - val_accuracy: 0.7648 - val_loss: 0.6690
73/73 ━━━━━━━━━━━━━━━━━━━━ 26s 359ms/step - accuracy: 0.7585 - loss:
0.6986
Validation Accuracy: 0.7648
```

Question 10: You are working on a web application for a medical imaging startup. Your task is to build and deploy a CNN model that classifies chest X-ray images into “Normal” and “Pneumonia” categories. Describe your end-to-end approach–from data preparation and model training to deploying the model as a web app using Streamlit.

```
pip install streamlit
```

```
Collecting streamlit
  Downloading streamlit-1.52.2-py3-none-any.whl.metadata (9.8 kB)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (5.5.0)
Requirement already satisfied: blinker<2,>=1.5.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: cachetools<7,>=4.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (6.2.4)
Requirement already satisfied: click<9,>=7.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (8.3.1)
Requirement already satisfied: numpy<3,>=1.23 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (25.0)
Requirement already satisfied: pandas<3,>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Requirement already satisfied: protobuf<7,>=3.20 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.5)
Requirement already satisfied: pyarrow>=7.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tenacity<10,>=8.1.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (9.1.2)
```

```
Requirement already satisfied: toml<2,>=0.10.1 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.4.0 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (4.15.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.45)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in
/usr/local/lib/python3.12/dist-packages (from streamlit) (6.5.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!
=5.4.1,<7,>=4.0->streamlit) (3.1.6)
Requirement already satisfied: jsonschema>=3.0 in
/usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!
=5.4.1,<7,>=4.0->streamlit) (4.25.1)
Requirement already satisfied: narwhals>=1.14.2 in
/usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!
=5.4.1,<7,>=4.0->streamlit) (2.13.0)
Requirement already satisfied: gitdb<5,>=4.0.1 in
/usr/local/lib/python3.12/dist-packages (from gitpython!
=3.1.19,<4,>=3.0.7->streamlit) (4.0.12)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0-
>streamlit) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0-
>streamlit) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0-
>streamlit) (2025.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27-
>streamlit) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27-
>streamlit) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27-
>streamlit) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27-
>streamlit) (2025.11.12)
Requirement already satisfied: smmap<6,>=3.0.1 in
/usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1-
>gitpython!=3.1.19,<4,>=3.0.7->streamlit) (5.0.2)
Requirement already satisfied: MarkupSafe>=2.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!
=5.4.1,<7,>=4.0->streamlit) (3.0.3)
Requirement already satisfied: attrs>=22.2.0 in
/usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!
=5.4.0,!=5.4.1,<7,>=4.0->streamlit) (25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!
=5.4.0,!=5.4.1,<7,>=4.0->streamlit) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!
=5.4.0,!=5.4.1,<7,>=4.0->streamlit) (0.37.0)
Requirement already satisfied: rpds-py>=0.7.1 in
/usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!
=5.4.0,!=5.4.1,<7,>=4.0->streamlit) (0.30.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas<3,>=1.4.0->streamlit) (1.17.0)
Downloading streamlit-1.52.2-py3-none-any.whl (9.0 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.0/9.0 MB 96.9 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 87.1 MB/s eta
0:00:00
lit
Successfully installed pydeck-0.9.1 streamlit-1.52.2
```

```python
# ============================================================
# STEP 0: SUPPRESS WARNINGS
# ============================================================
import os, warnings
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
warnings.filterwarnings("ignore")

# ============================================================
# STEP 1: IMPORT LIBRARIES
# ============================================================
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout

# ============================================================
# STEP 2: LOAD DATASET
# ============================================================
(ds_train, ds_val), ds_info = tfds.load(
    'cats_vs_dogs',
    split=['train[:10%]', 'train[10%:15%]'],  # small fractions for
demo
    as_supervised=True,
```

```
    with_info=True
)

def preprocess(image, label):
    image = tf.image.resize(image, (150,150)) / 255.0
    return image, label

train_data =
ds_train.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
val_data = ds_val.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)

# ============================================================
# STEP 3: BUILD CNN MODEL
# ============================================================
model = Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(150,150,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation="relu"),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation="relu"),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.5),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# ============================================================
# STEP 4: TRAIN MODEL
# ============================================================
history = model.fit(train_data, epochs=3, validation_data=val_data)

# Save trained model
model.save("cnn_model.h5")

# ============================================================
# STEP 5: STREAMLIT DEPLOYMENT
# ============================================================
# Run this section using: streamlit run filename.py
import streamlit as st
import numpy as np
```

```python
from PIL import Image

st.title("Cats vs Dogs Classifier (Demo)")

@st.cache_resource
def load_model():
    return tf.keras.models.load_model("cnn_model.h5")

model = load_model()

uploaded_file = st.file_uploader("Upload an Image",
type=["jpg","png"])
if uploaded_file:
    img = Image.open(uploaded_file).resize((150,150))
    img_array = np.array(img)/255.0
    img_array = img_array.reshape(1,150,150,3)

    prediction = model.predict(img_array)[0][0]
    label = "Dog" if prediction > 0.5 else "Cat"

    st.image(img, caption="Uploaded Image")
    st.subheader(f"Prediction: {label}")
    st.write(f"Confidence Score: {prediction:.2f}")
```

```
Epoch 1/3
73/73 ━━━━━━━━━━━━━━━━━━━━ 130s 2s/step - accuracy: 0.5030 - loss:
0.7616 - val_accuracy: 0.5632 - val_loss: 0.6870
Epoch 2/3
73/73 ━━━━━━━━━━━━━━━━━━━━ 121s 2s/step - accuracy: 0.5364 - loss:
0.6845 - val_accuracy: 0.5761 - val_loss: 0.6695
Epoch 3/3
73/73 ━━━━━━━━━━━━━━━━━━━━ 121s 2s/step - accuracy: 0.5921 - loss:
0.6704 - val_accuracy: 0.5632 - val_loss: 0.7503
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
2026-01-04 19:44:30.111 WARNING
streamlit.runtime.scriptrunner_utils.script_run_context: Thread
'MainThread': missing ScriptRunContext! This warning can be ignored
when running in bare mode.
2026-01-04 19:44:30.281
  Warning: to view this Streamlit app on a browser, run it with the
following
  command:

    streamlit run
/usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py
```

```
[ARGUMENTS]
2026-01-04 19:44:30.282 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.284 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.287 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.287 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.289 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.292 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
WARNING:absl:Compiled the loaded model, but the compiled metrics have
yet to be built. `model.compile_metrics` will be empty until you train
or evaluate the model.
2026-01-04 19:44:30.466 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.467 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.506 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.508 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.508 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.509 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.510 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.511 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
2026-01-04 19:44:30.511 Thread 'MainThread': missing ScriptRunContext!
This warning can be ignored when running in bare mode.
```